



DEPARTAMENTO DE INFORMÁTICA
UNIVERSIDAD CARLOS III DE MADRID

Bachelor in Applied Mathematics and Computing

Machine Learning I
Course 2024-2025

Tutorial 4: Introduction to Reinforcement Learning

April 7, 2026

- The aim of this tutorial is to understand the core concepts of Reinforcement Learning (*Q-learning*, exploration vs. exploitation, and stochastic environments) using the modern *Gymnasium* library.
- The tutorial can be done in Linux, Windows or Mac.
- It is important to solve the exercises in order.

Introduction

Reinforcement Learning (RL) is a paradigm of machine learning where an *agent* learns to make decisions by interacting with an *environment*. Unlike supervised learning, there is no labeled dataset providing the "correct" actions. Instead, the agent must discover which actions yield the highest cumulative reward through trial and error. This mathematical framework is formalized as a Markov Decision Process (MDP), defined by a set of states, actions, transition probabilities, and a reward function.

In this tutorial, you will implement **Q-Learning**, one of the most foundational model-free RL algorithms. The agent will learn a policy by updating a *Q-Table* that estimates the expected utility of taking a given action in a specific state, governed by the Bellman equation.

To bring this to life, we will use **Gymnasium** (pip install gymnasium). Gymnasium provides a unified API to standard RL environments. The chosen domain for this practice is *FrozenLake-v1*. The agent controls a character navigating a grid of walkable ice tiles and dangerous holes. The objective is to reach the goal without falling into the water. Through this exercise, you will explore the exploration-exploitation dilemma and analyze how an agent adapts to both deterministic and stochastic (slippery) environments.

1 Exercise 1: Basic Implementation and Execution Modes

The provided code *qlearning-frozenlake.py* features an execution mode variable (`MODE`) that allows you to switch between training the agent ("**TRAIN**") and watching a fully trained agent play ("**PLAY**").

Furthermore, the script saves the *Q-Table* at the end of each episode into a text file (`qtable.txt`). If you stop the execution and run it again in "**TRAIN**" mode, it will load this file and resume training from where it left off, rather than starting from scratch.

1. Open *qlearning-frozenlake.py* and ensure `MODE = "TRAIN"`.
2. Implement the Q-Learning update formula (**TODO 1**) and the Epsilon Decay (**TODO 2**).

3. Run the script. A file named `qtable.txt` will be created in your directory. Close the graph when the training finishes.
4. Now, change the configuration to `MODE = "PLAY"` and run the script again. Do not touch the code. Watch how the agent relies entirely on the `qtable.txt` file to make its decisions.

Questions

1. Open `qtable.txt` with a text editor. Explain what the rows and columns represent.
2. Identify the row corresponding to the state immediately to the left of the Goal. Which action has the highest value? Does this value make sense mathematically given the rewards?
3. If you delete `qtable.txt` and immediately run the script in "PLAY" mode, what behavior would you expect from the agent on the screen? Why?

2 Exercise 2: Stochastic Environments

Now we are going to introduce uncertainty into the agent's movements, transforming the problem into a stochastic Markov Decision Process (MDP).

1. In your `qlearning_frozenlake.py` script, change the environment initialization parameter to `is_slippery=True`.
2. Run the agent again with the exact same hyperparameters you used in Exercise 1.

Questions

1. What is going on with the agent's movements when `is_slippery=True`? Does the training converge as fast as in the deterministic case?
2. Look at the maximum average reward your agent achieves at the end of the stochastic training. Does it reach a solid 1.0 (100% success rate)? Explain why this happens given the nature of the slippery ice.
3. Analyze the new Q-table. Compare the Q-values of a safe cell that is immediately adjacent to a hole in both scenarios (deterministic vs. stochastic). Why might the optimal action in the stochastic scenario NOT be the shortest path to the goal?
4. Generate a heatmap (using `matplotlib` or `seaborn`) of the maximum Q-values for each state from your final Q-table. Some zones of the map will have values equal to 0.0. Is this because those states are dangerous, or because the agent never explored them? Justify your answer.

3 Files to submit

All the lab assignments **must** be done in groups of 2 people. You must submit a .zip file containing the required material through Aula Global before the following deadline: **Friday, April 14th at 17:00**. The name of the zip file must contain the last 6 digits of both student's NIA, i.e., `tutorial4-123456-234567.zip`

The zip file must contain the following files:

1. A **PDF** document with:
 - Cover page with the names and NIAs of both students.
 - Answers to all the questions from Exercise 1 and Exercise 2.
 - The training convergence graphs generated in both exercises.
 - The heatmap of the Q-table requested in Exercise 2.
 - Brief conclusions on the impact of stochasticity in Reinforcement Learning.
2. The completed Python script (`qlearning_frozenlake.py`).
3. The final Q-table file (`qtable.txt`) generated after the stochastic training.

Please, **be very careful and respect the submission rules**.